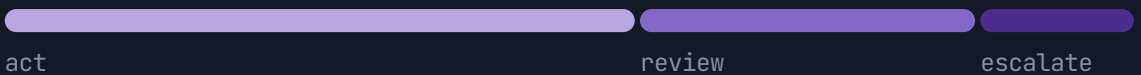


Decide, Don't Generate



Jev, System One Models, and the
Decision Layer of Agentic AI

Runnable Labs • 130+ figures • no API key needed

Sridhar Mukkandi

Table of contents

Preface	vii
How to read this book	ix
Prologue: the night shift	xii
I Prerequisites: probability and decisions	xv
1 What “learning” means	1
2 Probability, and how a machine learns it	13
3 Calibration: when 0.8 really means 80%	23
4 From probabilities to actions	34
II Prerequisites: networks, language models and agents	43
5 Deep learning in one chapter	45
6 How an LLM writes, and structured outputs	53
7 RAG, agents, and where they break	62
III Jev in depth	71
8 System 1 and System 2	73
9 Inside Jev: what we know and what we don’t	81
10 The type system: choice, score, noul	90
11 Testing Jev’s calibration yourself	98
12 The Jevons paradox of decisions	107
IV The decision layer	117
13 The bake-off: six ways to make a decision	119

14 Act, review, or escalate	129
15 A catalogue of decision patterns	143
V Building	155
16 First calls, and the mock that makes them free	157
17 A hybrid agent: Jev decides, the LLM reasons	166
18 Case study: SOC alert triage	174
19 An applications gallery	182
20 Build your own System One model	190
21 Capstone: a production decision service	199
VI What changes now	209
22 What changes now	211
Appendices	218
Glossary	218
Cheat sheets	222
Bibliography	225
Index	229
About the Author	231

Calibration: when 0.8 really means 80%

When you can measure what you are speaking about, and express it in numbers, you know something about it.

— Lord Kelvin, lecture on electrical units of measurement, 1883

A question that sounds silly until it costs you money:

“If a model says 80%, does that mean 80%?”

Your instinct is probably “obviously”: it’s a probability, and that’s what the number is for.

But a number is just a number. A model can say 0.8 about alerts that turn out to be attacks half the time, or nineteen times out of twenty. Nothing in the maths of the model forces its 0.8 to line up with reality. It lines up only if we built it well, trained it on the right data and checked, and even then it can drift.

This chapter is about checking, the single most important habit in the book. Every threshold in Chapter 4, every policy in Chapter 14 and every claim about Jev in Part III rests on it.

BY THE END OF THIS CHAPTER

- Say what it means for a model to be *calibrated*, and why that’s different from being accurate.
- Draw and read a reliability diagram, and compute the expected calibration error (ECE).
- Recognise the everyday mistakes that break calibration, including rebalancing your training data.
- Fix a miscalibrated model with Platt scaling, temperature scaling or isotonic regression, and know which to use.
- Check calibration within the groups you’ll actually act on.

The weather forecaster's secret

Let's go back to the weather.

People love to complain about forecasters. But in the 1970s, when researchers checked American weather forecasters' chance-of-rain forecasts against what actually happened, they found something surprising. When the forecasters said "30% chance of rain", it rained on roughly 30% of those days. When they said 70%, it rained roughly 70% of the time. Across the whole range, what they said and what happened lined up almost perfectly (Murphy and Winkler 1977).

Compare that with how people usually do. Ask someone to give a range they're "90% sure" contains the answer to a trivia question, and the truth lands inside far less often than 90% of the time. We are, as a species, reliably too sure of ourselves.

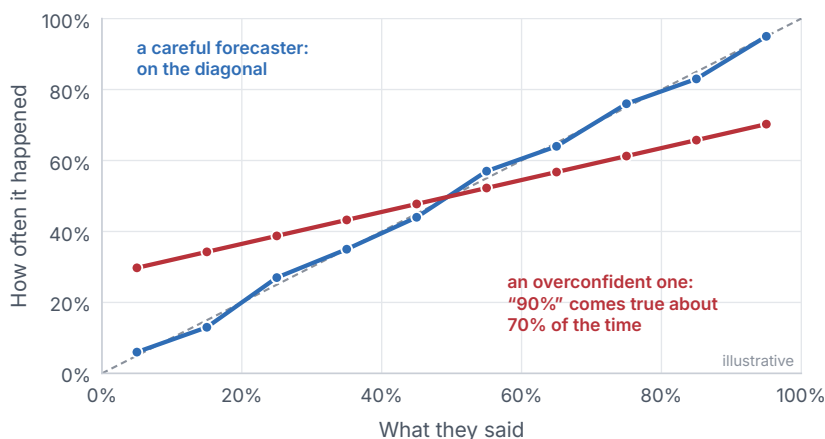


Figure 3.1. Two forecasters. For each thing they said ("30%", "70%" and so on), we count how often it actually happened. The careful forecaster sits on the diagonal: their 70% happens about 70% of the time. The overconfident one says 90% about things that happen only about 70% of the time.

What makes the weather forecasters good isn't that they're always right. They can't be. It's that their numbers mean what they say. That property has a name.

A model is calibrated when its 80% happens about 80% of the time.

And, importantly, calibration is something you can *check*, with nothing more than grouping and counting. You collect everything the model said "about 80%" to, and count how often it happened.

Good at ranking, bad at honesty

Before we check anything, one idea needs to land, because people confuse it constantly.

There are two separate things you might want from a model's probabilities.

The first is **ranking**: does the model give attacks higher numbers than harmless alerts? If you sort alerts by the model's score, do the real attacks rise to the top? The AUC measures this, the number you've seen in a few tables already. An AUC of 1 means every attack scored above every harmless alert. An AUC of 0.5 means the scores are no better than a coin flip.

The second is **honesty**: when the model says 0.3, do three in ten of those alerts turn out to be attacks? That's calibration.

The two are independent. A model can rank perfectly and still lie about how sure it is.

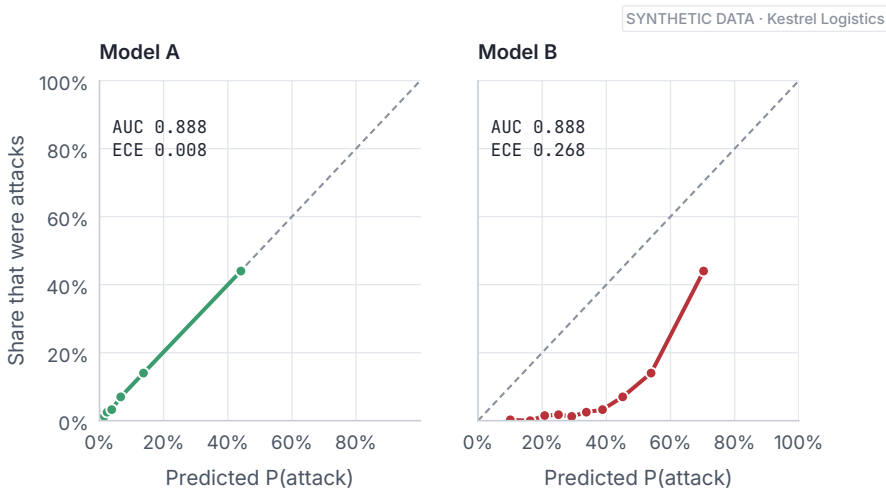


Figure 3.2. Two models, identical at ranking (same AUC) but not at calibration. Model A is the logistic regression from Chapter 2. Model B is the same model with its probabilities distorted: it still sorts the alerts in the same order, but its numbers no longer mean anything.

Figure 3.2 shows two models with the same AUC, 0.888. Model B's numbers are a mess: it says 40% about alerts that are attacks only a few percent of the time. If you only ever sort alerts, Model B is as good as Model A. The moment you put a threshold on its numbers, or add them into a cost, or show them to an analyst, it'll mislead you.

Here lies the trap: most leaderboards and most model comparisons report ranking metrics like AUC, accuracy or F1. Calibration is often not measured at all. For a decision system it's usually the thing that matters most.

The reliability diagram

So how do we check? The same way the weather researchers did.

1. Take a pile of predictions where you know what happened: our test alerts from Chapter 2.
2. Sort them into bins by what the model said: 0–10%, 10–20% and so on.
3. In each bin, compare two numbers: the model's *average* prediction and the share of alerts that really were attacks.
4. Plot one against the other.

That plot is called a **reliability diagram**, and you'll see a lot of them from here on.

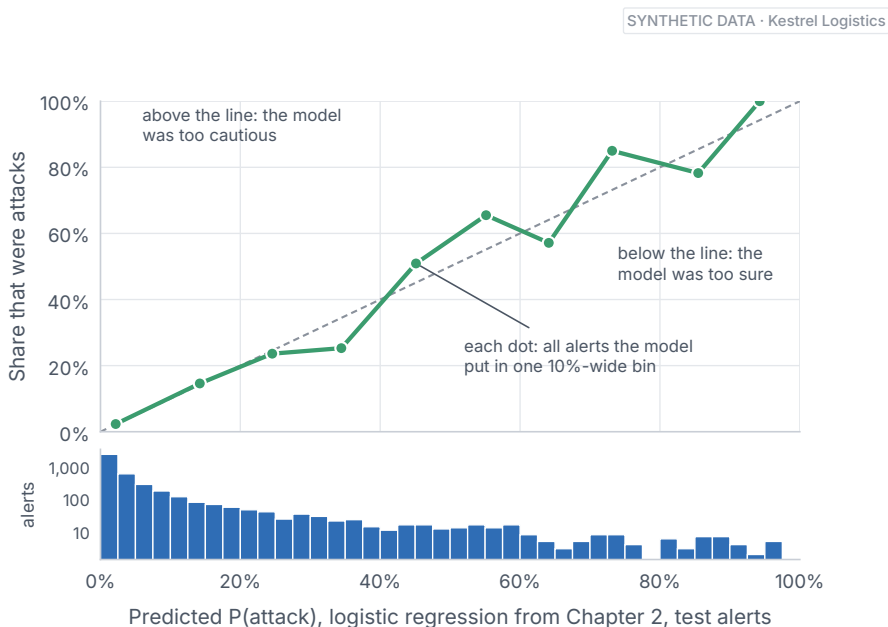


Figure 3.3. The reliability diagram of our Chapter 2 model on alerts it never saw. Points on the dashed diagonal mean the model's numbers can be taken at face value. Underneath, a histogram (log scale) shows how many alerts fall in each part of the range: most sit near zero, and only a handful above 50%.

Look at Figure 3.3 carefully, top and bottom together, because the bottom panel is the one people forget. Our model's points sit near the diagonal where there's lots of data. Up at the right end, the dots jump around, and the histogram tells you why: only a few dozen alerts live up there. A wobbly dot with twelve alerts behind it is weak evidence, as Chapter 2 warned. Always look at the counts before you panic, or relax.

 TRY IT

Build the diagram's table yourself: bin, average prediction, observed rate, count.

```
alerts = soc.load()
train, calib, test = soc.split(alerts)
F = lambda d: soc.feature_matrix(d).to_numpy()
model = LogisticRegression(C=1e4, max_iter=5000).fit(
    F(train), train.malicious)
p = model.predict_proba(F(test))[:, 1]
y = test.malicious.to_numpy()

bins = np.clip((p * 10).astype(int), 0, 9)
table = (pd.DataFrame({"bin": bins, "said": p, "happened": y})
        .groupby("bin")
        .agg(said=("said", "mean"), happened=("happened", "mean"),
            n=("said", "size")))
print(table.round(3).head(6))
```

	said	happened	n
bin			
0	0.022	0.023	3266
1	0.142	0.146	315
2	0.246	0.236	144
3	0.344	0.253	87
4	0.451	0.509	55
5	0.551	0.655	58

One number: ECE

Pictures are the best way to *see* calibration, but sometimes you need a single number, to compare models or to raise an alarm when it drifts. The most common is the **expected calibration error**, ECE: the average gap between “said” and “happened” across the bins, weighted by how many predictions each bin holds (Naeini et al. 2015).

```
print(f"ECE: {cal.ece(p, y):.4f}")
```

```
ECE: 0.0077
```

Our logistic regression's ECE is 0.008, meaning that on average, its probabilities are off by less than one percentage point. Good. Model B's is 0.27.

ECE has weaknesses worth knowing. It depends on how you choose the bins. And because it's an average, a model can hide terrible calibration in a

small region, such as the high end where the dangerous alerts live, behind excellent calibration where most of the data sits. Report it, but always look at the picture too.

🔍 GOING DEEPER optional

The Brier score from Chapter 2 splits neatly into three parts, a result due to Allan Murphy (Murphy 1973): $\text{Brier} = \text{reliability} - \text{resolution} + \text{uncertainty}$. **Uncertainty** is how hard the problem is to begin with (it depends only on the base rate).

Resolution is how much the model's predictions separate cases that turn out differently, which is roughly the ranking skill. **Reliability** is the calibration error: the squared gap between said and happened, bin by bin. For our model: reliability 0.0005, resolution 0.0212, uncertainty 0.0698. A model can improve its Brier score by getting better at ranking, or by getting better calibrated, and the decomposition tells you which.

How calibration breaks

If log loss rewards honesty, as Chapter 2 said, why would a model ever come out miscalibrated? Because the training setup can mislead it. Here are the usual suspects.

Rebalancing the data. This is one of the most common, and it's usually done with the best intentions. Attacks are rare, only 8% of alerts, so someone decides the model needs to “see more attacks” and trains it on a 50/50 mix: every attack, plus an equal number of randomly chosen harmless alerts. The ranking survives. But the model has learned from a world where attacks are *common*, and it faithfully reports probabilities for that world.

The rebalanced model's average prediction is 29%. The real attack rate is 8%. Put a threshold on those numbers and you'll drown your analysts.

Overfitting. Flexible models that memorise, like the deep decision tree from Chapter 2, push their probabilities towards 0 and 1, because on the training data they really were that sure. They come out overconfident on new data.

Scores that were never probabilities. Some models output a “score” that happens to fall between 0 and 1 but was never trained with a proper scoring rule. Treat it as a ranking until you've checked it.

Big modern networks. Surprisingly, very large neural networks trained for classification tend to be *overconfident* (Guo et al. 2017). Chapter 5 looks at why. And the “confidence” an LLM states in words is a different animal again. Chapter 6 measures it.



Figure 3.4. A model trained on 50/50 rebalanced data (red) thinks attacks are about ten times more common than they are, and its probabilities sit far below the diagonal. After Platt scaling fitted on a separate calibration set (green), the same model is well calibrated again.

The world changing. A model calibrated on last month’s base rate is miscalibrated the day a campaign doubles the number of attacks. Chapter 14 will watch it happen.

Fixing it

The good news: a model with good ranking and bad calibration is usually easy to fix. You don’t retrain it. You put a small correction on top.

The recipe is always the same. Hold back a **calibration set**, data the model never trained on. That’s the 20% we set aside in Chapter 2. Run the model on it, and learn a simple function that maps the model’s raw probabilities to calibrated ones. Then check the result on the *test* set, which neither the model nor the correction has seen.

There are three common corrections, and they differ in how flexible they are.

Platt scaling fits a tiny logistic regression on top of the model’s log-odds: two numbers, a stretch and a shift (Platt 1999). It’s the usual first choice.

Temperature scaling fits just one number, T , and divides the log-odds by it (Guo et al. 2017). T above 1 softens overconfident predictions; below 1 sharpens timid ones. It’s the standard fix for overconfident neural networks, and it’s what people usually mean when they talk about calibrating an LLM classifier.

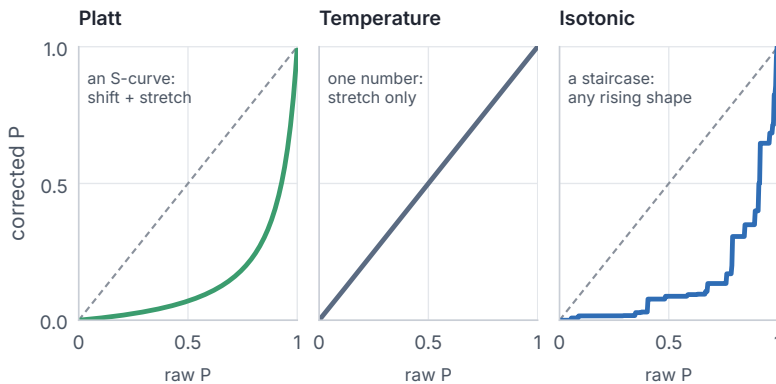


Figure 3.5. The three corrections, each fitted to the rebalanced model on the calibration set. Platt scaling learns an S-curve that can shift and stretch. Temperature scaling learns one number that only stretches around the middle, and here it barely changes anything. Isotonic regression learns any rising staircase.

Isotonic regression fits a staircase that only ever goes up (Zadrozny and Elkan 2002). It can fix almost any shape, but it needs more data and can overfit a small calibration set.

```
pos = train[train.malicious == 1]
neg = train[train.malicious == 0].sample(len(pos), random_state=0)
bal = pd.concat([pos, neg])
model_b = LogisticRegression(C=1e4, max_iter=5000).fit(
    F(bal), bal.malicious)
pb = model_b.predict_proba(F(test))[:, 1]
pb_cal = model_b.predict_proba(F(calib))[:, 1]

print(f'{"no fix":>12}: ECE {cal.ece(pb, y):.3f}')
for name, fixer in [("Platt", cal.Platt()),
                    ("Temperature", cal.Temperature()),
                    ("Isotonic", cal.Isotonic())]:
    fixer.fit(pb_cal, calib.malicious)
    print(f'{"name":>12}: ECE {cal.ece(fixer(pb), y):.3f}')
```

```
no fix: ECE 0.216
Platt: ECE 0.010
Temperature: ECE 0.215
Isotonic: ECE 0.013
```

Platt and isotonic fix the rebalanced model almost completely. Temperature scaling doesn't help at all, and the reason teaches you something. Rebalancing didn't make the model overconfident. It *shifted* every prediction upwards, as if attacks were common. A temperature can only stretch or squeeze predictions around 0.5. It can't slide them all down. Platt can, because it has a shift as well as a stretch.

Temperature fixes overconfidence. It can't fix a wrong base rate.

Honest on average, dishonest in the corner

One more trap, and it's the one that bites in production.

A model can be well calibrated *overall* and badly calibrated for a particular group. Take a version of our model trained without knowing which detection rule fired, so it can't tell an email alert from an endpoint one.

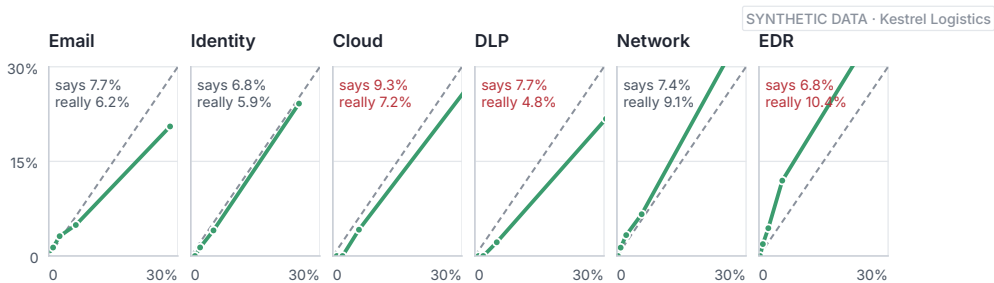


Figure 3.6. Calibration checked separately for each alert source. Across all sources together, this model is well calibrated, with an ECE of about half a percentage point. But it overestimates the risk of DLP and Cloud alerts and underestimates EDR alerts, which really are more dangerous here. Averaging hides both mistakes.

Its overall ECE is excellent, 0.005. Yet for endpoint (EDR) alerts it says 6.8% when the truth is 10.4%, and for DLP alerts it says 7.7% when the truth is 4.8%. The over- and under-estimates cancel out in the average.

If your policy treats every source the same way, you'll send too many harmless DLP alerts to the queue and close too many real EDR ones. The rule is simple: **check calibration inside every group you'll act on differently**, and inside every group where a mistake would be especially costly. Sources, customer tiers, languages, regions, and yes, demographic groups when a model touches people's lives.

SET THE THRESHOLD

Kestrel auto-closes any alert under 3%. The model above says EDR alerts average 6.8% risk, but they're really 10.4%. Roughly how much more often than planned will a real EDR threat be auto-closed? What are two different ways to fix this: one that changes the model, and one that changes only the policy?

(About half as often again. You could add the source as a feature, or calibrate separately per source, so the model's numbers hold up for every source. Or you could keep the model and set a stricter act line for EDR alerts. The first is better; the second is quicker.)

⚠ WHERE THIS BREAKS

Calibration is always measured *on some data*, and it's only guaranteed for data like that. A model calibrated on last quarter's alerts can drift as tools, attackers and staff change. Isotonic regression fitted on a few hundred examples will overfit and produce jagged, overconfident corrections. And calibration says nothing about any *single* prediction: a perfectly calibrated 70% is still wrong three times in ten. Keep a fresh calibration set, refit on a schedule, and keep watching the diagram.

“

The first principle is that you must not fool yourself, and you are the easiest person to fool.

— Richard Feynman, Cargo Cult Science, 1974

Where we are

The weather forecasters showed us that honesty is checkable. Then two models with the same ranking and very different calibration showed us why checking matters. Grouping and counting gave us the reliability diagram, and a well-meant rebalancing broke it. A small correction fitted on held-out data put it back. And just when the average looked fine, the EDR alerts reminded us to look inside every group we'll act on.

When Part III tests Jev's own calibration, it'll use these same tools, pointed at the mock, with everything in this chapter as the yardstick.

EXERCISES

1. Explain to a colleague, without jargon, the difference between a model that ranks well and a model that's calibrated. Use a weather example.
 2. In the lab, train the rebalanced model with a 30/70 mix instead of 50/50. Is it more or less miscalibrated? Predict the answer before you run it.
 3. Why must the calibration correction be fitted on a separate calibration set, not on the training set? What would go wrong?
 4. Fit isotonic regression using only 200 alerts from the calibration set. Plot its staircase. What do you notice, and what does that tell you about when to prefer Platt?
 5. Name three groups in a system you know where calibration might differ, and one decision in each that would go wrong if it did.
-

Next: we finally have probabilities we can trust. Chapter 4 turns them into actions: what each mistake costs, where the lines go, and how to see the whole trade-off on one page.

The bake-off: six ways to make a decision

There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies.

— C. A. R. Hoare, Turing Award lecture, 1980

Every SOC has the same argument, and it never quite ends.

The senior analyst says the rules are fine; they're written by people who know the attacks, and you can read every one. The data scientist says a trained model would beat the rules, if only someone would label enough alerts. The new engineer has wired an LLM into the ticketing system and says it reads alerts better than either of them. And now someone has read about Jev.

Each of them is right about something. The argument never ends because they're scoring on different things: one on explainability, one on accuracy, one on flexibility, and one on price. Nobody puts all the numbers on one sheet.

So let's do that. Six methods, one dataset, one fair contest, and a scorecard with every column that matters. I'll tell you now that Jev doesn't win every round.

BY THE END OF THIS CHAPTER

- Compare rules, logistic regression, a trained text classifier, LLM structured output, LLM-as-judge and Jev on the same decision.
- Score them on ranking, calibration, labels needed, speed, price and stability, not on one number.
- See where Jev loses, and why that's useful to know.
- Tell which results come from how the synthetic contestants were built, and which would carry over to your own test.
- Pick a method, or a combination, from a scorecard.

The contestants

The decision is Kestrel’s: is this alert a real threat? Every method gets the three history weeks, 14,973 alerts with analysts’ verdicts, to train or tune on. Every method is then scored on the fourth week, 5,027 alerts it has never seen, of which 412 are real.

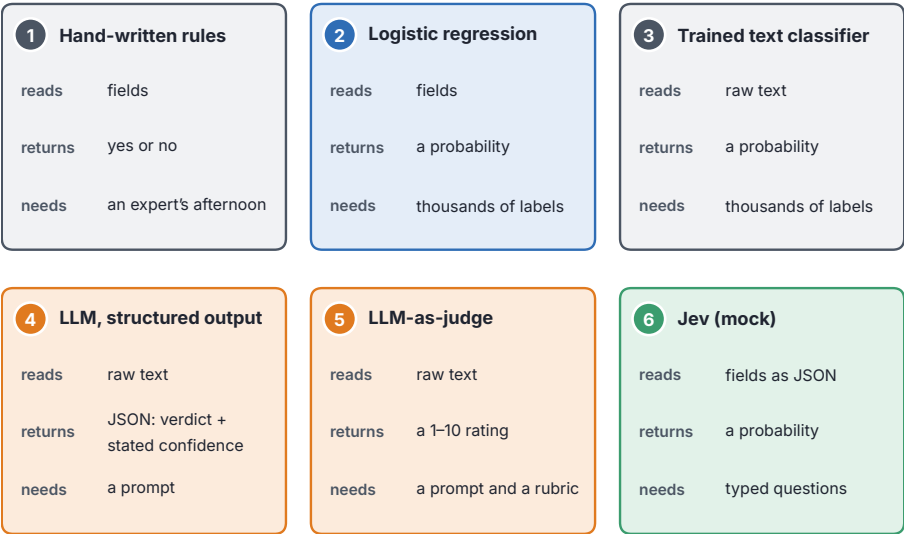


Figure 13.1. The six contestants: what each one reads, what it returns, and what it needs before it can start.

Each reads what it would naturally read in practice (Figure 13.1). The rules and the logistic regression get the alert’s structured fields from the SIEM. The text classifier and both LLM methods get the raw alert text. Jev gets the fields as a JSON state, the same information the logistic regression sees.

The **text classifier** needs a word of explanation. It stands in for a fine-tuned model: a classic word-counting classifier (word weights plus logistic regression) trained on the history labels, which re-trains in seconds on any laptop. A fine-tuned transformer, even a small one of the MiniLM kind, would usually read the text better, so treat this row as a floor for what a trained text model can do.

One warning before any results, and it’s the most important paragraph in this chapter. Both LLM contestants are the book’s mock, and I built it as a *noisier reader* than mock Jev (the repository’s docs/mock-design.md explains how). So when the LLM ranks alerts worse than Jev below, that’s a consequence of how the mocks were made. It isn’t evidence about real LLMs or real Jev. What *does* carry over is the shape of the other results: how stated confidences behave, how often free-text JSON breaks, what happens when

you ask twice, what labels cost. Those come from how each kind of method works, not from my choice of noise.

Round one: ranking

The first question is the one people usually ask: which method puts the real threats at the top?

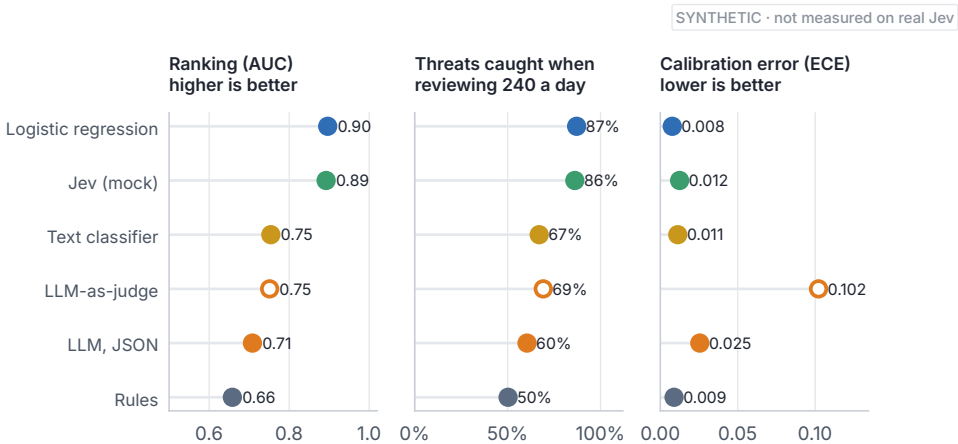


Figure 13.2. Three views of how well each method separates threats from harmless alerts, on the live week. Left: ranking quality (AUC). Middle: the share of all real threats found if the team reviews the top 240 alerts each day. Right: calibration error, where lower is better.

Logistic regression wins round one, narrowly: an AUC of 0.896 against Jev’s 0.892 (Figure 13.2). With Kestrel’s review capacity of 240 alerts a day, it catches 87% of real threats; Jev catches 86%.

That isn’t a fluke. The logistic regression was trained on fifteen thousand of Kestrel’s own labelled alerts, on clean structured fields, for a decision that’s nearly linear in those fields. Classic machine learning is at home here. A general model reading the same fields with no training on Kestrel at all shouldn’t be expected to beat it there, and it doesn’t.

The rules come last on ranking, with an AUC of 0.66, for a structural reason: a rule says yes or no. Among the hundreds of alerts it says yes to, it has no way to say which to look at first.

Give Jev the raw text instead of the fields, and its AUC drops to 0.856. Still well ahead of the trained text classifier’s 0.754, because reading “threat intel score: 0.82” as a number is what a word-counting classifier is worst at.

Round two: honesty

Ranking is only half the job. Chapter 14 will draw lines at probabilities, and lines only work if the probabilities mean what they say.

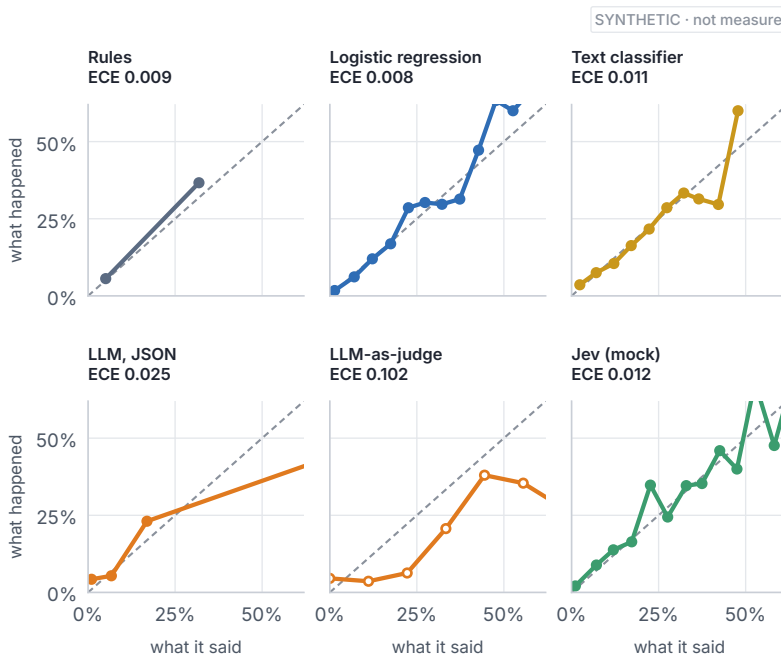


Figure 13.3. Reliability diagrams for each method on the live week. On the dashed line, what a method said matches what happened. Most methods here are close. The LLM's stated confidences cluster on a few values; the judge's ratings, read as probabilities, are far too high in the middle.

Most of the contestants are close to the line (Figure 13.3). The logistic regression was fitted with log loss on this company's data, the recipe Chapter 2 said produces honest probabilities. Mock Jev was designed to be well calibrated at Kestrel, and Chapter 11 showed that's worth checking at your own company, not assuming.

The two LLM methods are the interesting ones.

The LLM's stated confidence came from the JSON it wrote: "confidence": 0.95. Across five thousand alerts, it used only 6 distinct values, mostly 0.9, 0.95 and 0.99. Chapter 6 predicted as much. A number that a model *writes* is text, and text tends to be round. It can't rank finely, which is why the LLM's AUC is the lowest of the probability methods.

The judge's 1-to-10 rating was never meant to be a probability, and reading it as one gives the worst calibration in the contest: 0.102. You could fix that with Platt scaling on a few hundred labels, as Chapter 3 showed. But then you're no longer comparing a zero-label method.

Round three: how many labels?

“Logistic regression wins” comes with a price tag: fifteen thousand labels. What if you have fewer?

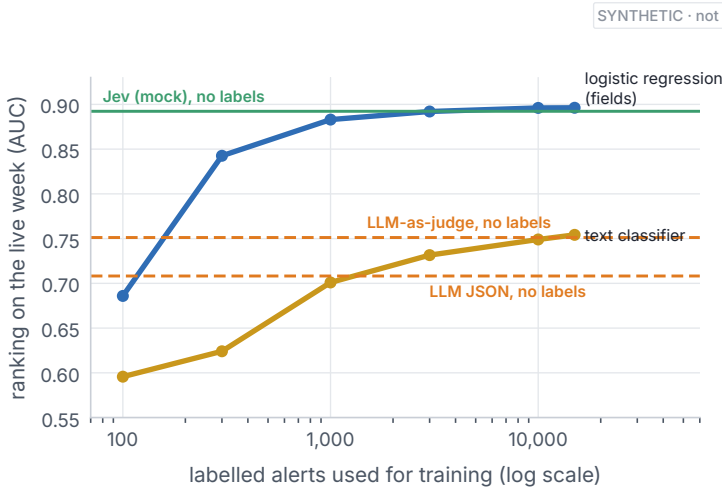


Figure 13.4. Ranking on the live week as the trained methods see more labelled alerts, averaged over random samples. Jev and the LLMs need no labels to start, so they’re flat lines. Logistic regression draws level with Jev at about 3,000 labels.

Figure 13.4 is the chart I’d show the data scientist in that argument. Logistic regression needs about 3,000 labelled alerts to draw level with Jev, and at a few hundred it’s clearly behind.

Where would 3,000 labels come from? Not from analysts’ reviews alone: at 240 a day, that’s nearly two weeks of verdicts. Kestrel’s labels are *eventual outcomes*: the incident that was confirmed or ruled out, the user who said “that was me”, the alert nobody followed up and nothing came of. Every alert gets one sooner or later, so at about 700 alerts a day, 3,000 labels is around four days of history, which Kestrel happens to have. A team that’s just starting, or a new decision nobody has labelled yet, doesn’t.

The text classifier never catches up, even with every label we have. Words alone, counted, don’t carry the numbers.

Labels are the hidden cost of trained models. A method that needs none can start today; a method that needs thousands has to wait for them, and needs them again when things change.

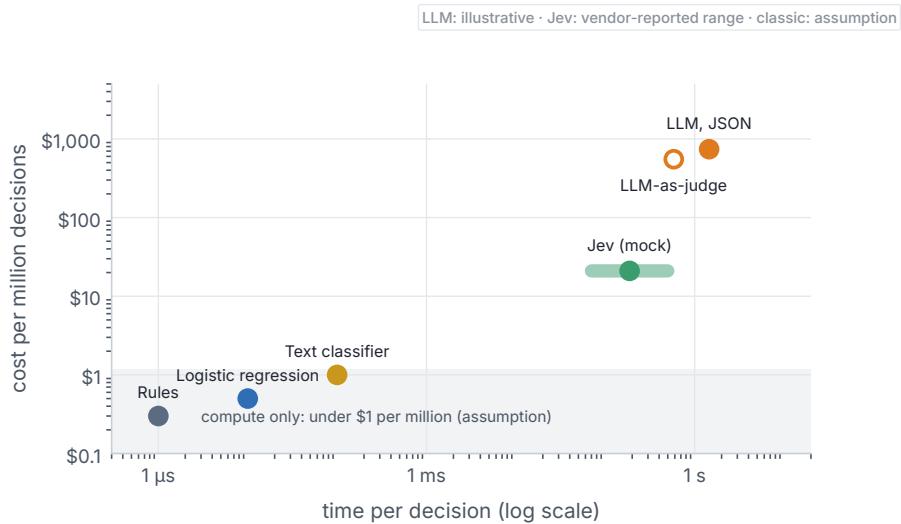


Figure 13.5. Time and cost per decision for each method, on log scales. Classic models run in microseconds for fractions of a cent per million (an assumption: compute only). The LLM figures are this book’s illustrative assumptions; Jev’s are vendor-reported.

Round four: speed and price

Nothing beats a rule on speed or price, and the classic models are close behind (Figure 13.5). They run in microseconds, on hardware you already own. Jev’s vendor-reported figures put it in between: about \$21 per million decisions, in tens to hundreds of milliseconds. The illustrative LLM costs about \$740 per million and takes about 1.4 seconds. A little more than Chapter 9’s figures, because the bake-off’s JSON answer carries four fields (verdict, confidence, rule and threat-intel score), about 60 output tokens rather than 40.

If a decision can be made well by a rule or a logistic regression, one of those is the cheapest, fastest option, by orders of magnitude. Jev’s price advantage is against *LLMs*, not against everything.

Round five: the same answer twice

Ask the LLM the same question five times at a temperature of 0.7, and 5.2% of verdicts change at least once (Figure 13.6). At temperature 0 the mock is repeatable, and real APIs mostly are too, though not always perfectly. And 3.2% of its JSON answers were broken badly enough to fail parsing. We fell back to the base rate for those, but in a real system each one is an exception to handle.

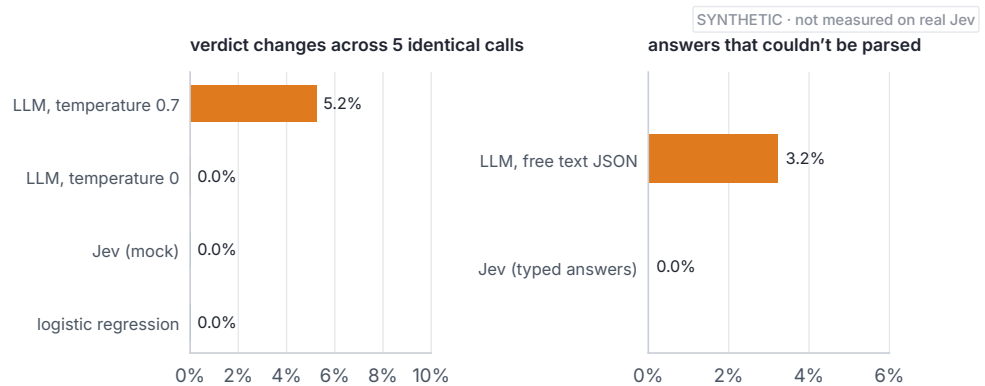


Figure 13.6. Left: the share of alerts whose verdict changed across five identical calls. Right: the share of answers that couldn't be parsed at all.

Jev’s typed answers can’t fail to parse: the probabilities arrive as numbers in a fixed shape. The mock always gives the same answer to the same question. Whether real Jev does is something to check.

The scorecard

Now put everything on one sheet.

SYNTHETIC · not measured on real Jev

	ranking (AUC)	caught at 240/day	calibration (ECE)	cost per million	time per decision	labels needed	same answer twice?	explains itself?
Rules	0.66	50%	0.009	< \$1	1 μs	none	yes	fully
Logistic regression	0.90	87%	0.008	< \$1	10 μs	~3,000	yes	weights
Text classifier	0.75	67%	0.011	\$1	100 μs	15,000+	yes	word weights
LLM, JSON	0.71	60%	0.025	\$740	1.4 s	none	not always	in words*
LLM-as-judge	0.75	69%	0.102	\$552	583 ms	none	not always	in words*
Jev (mock)	0.89	86%	0.012	\$21†	70–500 ms*	none‡	yes (mock)	no

Figure 13.7. The whole contest on one sheet. Bold marks the best in each numeric column. Every accuracy number is synthetic; every cost and time is an assumption, an illustration, or vendor-reported, as marked.

Read Figure 13.7 by column, not by row, and ask which columns *your* decision cares about.

If you have thousands of labels, clean fields and a decision that doesn't change much, logistic regression is hard to beat: it ranks best, it's calibrated, it's nearly free, and you can read its weights. Jev loses there, and plainly.

If you need to explain every decision to an auditor, rules win the last column outright, and nothing else comes close.

If you have no labels, raw text, or many different decisions to make, Jev's row is the strongest. It starts with zero labels, ranks nearly as well as the trained model, gives probabilities you can draw lines on after a check, answers in a shape that can't break, and costs a small fraction of an LLM call.

And the LLM? In this contest it's the weakest decider, partly by construction. But notice what it's good at that no other row even attempts: it reads anything, and it *writes*. That's why Chapter 15's first pattern gives it the job of extracting and explaining, and gives the decision to something else.

TRY IT

```
from jevkit import bakeoff as B    # the six contestants, one live week
r = B.run()
for m in B.METHODS:
    s = B.scores(r["probs"][m], r["y"])
    print(f"{B.NAMES[m]:>26}: AUC {s['auc']:.3f}, "
          f"ECE {s['ece']:.3f}")
```

```
Hand-written rules: AUC 0.658, ECE 0.009
Logistic regression: AUC 0.896, ECE 0.008
Trained text classifier: AUC 0.754, ECE 0.011
LLM, structured output: AUC 0.708, ECE 0.025
LLM-as-judge: AUC 0.751, ECE 0.102
Jev (mock): AUC 0.892, ECE 0.012
```

WHERE THIS BREAKS

This is one decision, on one synthetic company, with mocks standing in for the LLM and for Jev. The LLM-versus-Jev accuracy gap was built into the mocks and tells you nothing about the real models. The text classifier is a weak stand-in for a fine-tuned model. And a single live week has only about four hundred real threats, so differences in the second decimal place of AUC are within noise. The value of a bake-off is the *method*: run the same contest on your own data, with the real models, and fill in your own scorecard.

✦ SET THE THRESHOLD

A new team at Kestrel is taking over cloud-storage alerts, a decision nobody has labelled. They'll get about 300 labels a week from their own reviews. Which method would you start with, and when would you consider switching? What would you use the first 300 labels for?

(Start with a zero-label method, Jev in this contest, and use the first 300 labels to check and fix its calibration, as in Chapter 11. After three or four weeks of labels, train a logistic regression alongside it and compare on the next week. If the two agree closely, keep the cheaper one. If they disagree, look at the alerts where they do.)



For any algorithm, any elevated performance over one class of problems is offset by performance over another class.

— David Wolpert and William Macready, No Free Lunch Theorems for Optimization, 1997

Wolpert and Macready proved that about search and optimisation (Wolpert and Macready 1997), but it's the right spirit for any bake-off. There is no best method, only a best method for a decision.

Where we are

Six methods sat the same exam. The trained logistic regression won on ranking and calibration, because it had fifteen thousand labels on clean fields. It needed about three thousand of them just to draw level with a method that needed none. The rules won on explanation and price. The LLM's stated confidences turned out to be round numbers, and its free-text JSON broke often enough to matter. Jev's row was the strongest wherever labels were scarce or the input was raw text. None of that tells you what real models will do on your data. It tells you which columns to measure when you try.

EXERCISES

1. In the lab, write a better hand-written rule. How close to the logistic regression can you get? What did you have to add?
2. Fit Platt scaling to the LLM-as-judge ratings using 300 history alerts. What happens to its ECE? To its AUC? Why does one change and not the other?
3. Rerun the labels curve with the text classifier using bigrams only. Does it help? Why might a fine-tuned transformer do better?

4. Build a scorecard for a decision in your own work. Which three columns matter most? Which method would you try first?

Next: a probability isn't an action. Chapter 14 turns these numbers into three doors, act, review and escalate, and puts the lines where the costs say they belong.



That's the end of the sample.

The full book has 22 chapters, runnable labs for every one of them, and a free mock of Jev that needs no API key.

Code, labs and figures

<https://github.com/Mukkandi-Sridhar/JEVBook>

Contact

`sridhar.authorhub@gmail.com`